Practical 06

Aim: Considered there are N philosophers seated around a circular table with onebetween each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both. Write a program to solve the problem using process synchronization technique.

~

GNU nano 8.7 dining.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5   // Number of philosophers

#define THINKING 0
#define HUNGRY 1
#define EATING 2

int state[N];
int phil[N] = {0, 1, 2, 3, 4};

sem_t mutex;
sem_t S[N];

void test(int i) {
    if (state[i] == HUNGRY &&
        state[(i + 4) % N] != EATING &&
        state[(i + 1) % N] != EATING) {

        state[i] = EATING;
        sleep(1);
        printf("Philosopher %d takes chopsticks %d and %d\n",
               i + 1, (i + 4) % N + 1, i + 1);
        printf("Philosopher %d is Eating\n", i + 1);

        sem_post(&S[i]);
    }
}

void take_chopstick(int i) {
    sem_wait(&mutex);

    state[i] = HUNGRY;
    printf("Philosopher %d is Hungry\n", i + 1);

    test(i);

    sem_post(&mutex);
    sem_wait(&S[i]);
}

void put_chopstick(int i) {
    sem_wait(&mutex);

    state[i] = THINKING;
    printf("Philosopher %d putting chopsticks %d and %d down\n",
```

```
  GNU nano 8.7

    sem_post(&mutex);
    sem_wait(&S[i]);
}

void put_chopstick(int i) {
    sem_wait(&mutex);

    state[i] = THINKING;
    printf("Philosopher %d putting chopsticks %d and %d down\n",
           i + 1, (i + 4) % N + 1, i + 1);
    printf("Philosopher %d is Thinking\n", i + 1);

    test((i + 4) % N);
    test((i + 1) % N);

    sem_post(&mutex);
}

void* philosopher(void* num) {
    while (1) {
        int i = *(int*)num;
        sleep(1);

        take_chopstick(i);
        sleep(2);
        put_chopstick(i);
    }
}

int main() {
    int i;
    pthread_t thread_id[N];

    sem_init(&mutex, 0, 1);

    for (i = 0; i < N; i++)
        sem_init(&S[i], 0, 0);

    for (i = 0; i < N; i++) {
        pthread_create(&thread_id[i], NULL,
                       philosopher, &phil[i]);
        printf("Philosopher %d is Thinking\n", i + 1);
    }

    for (i = 0; i < N; i++)
        pthread_join(thread_id[i], NULL);
}
```

```
PRACHI@LAPTOP-CVJL7P0G UCRT64 ~
$ nano chopstick.c

PRACHI@LAPTOP-CVJL7P0G UCRT64 ~
$

PRACHI@LAPTOP-CVJL7P0G UCRT64 ~
$ gcc chopstick.c -lpthread -o chopstick
./chopstick
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 2 is eating
Philosopher 4 is eating
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 4 finished eating
Philosopher 2 finished eating
Philosopher 3 finished eating
Philosopher 0 is eating
Philosopher 1 finished eating
Philosopher 0 finished eating

PRACHI@LAPTOP-CVJL7P0G UCRT64 ~
```